AWS CLOUD INFRASTRUCTURE REPORT

Blogging Platform

Production Multi-Tier Cloud Architecture & Resource Directory

Amazon Web Services • Region: ap-southeast-2 (Sydney)

Student Details

Student Name:	Hardik Kumar
Registration Number:	24BCE7727
Slot:	B2

Live Application Load Balancer Endpoint:

http://astro-blog-alb-806893944.ap-southeast-2.elb.amazonaws.com

Table of Contents

AWS Cloud Infrastructure Resource Directory

#	Resource / Section	Description	Location
1	**Architecture Overview**	Production Multi-Tier Cloud Topology Diagram	**Page 3**
2	**Amazon EC2 & Auto Scaling**	Elastic Compute Instances & Auto Scaling Group	**Page 4**
3	**Application Load Balancer**	Layer 7 HTTP Load Balancer & Path Routing Rules	**Page 5**
4	**Amazon ECS Fargate Cluster**	Serverless Container Service for Comments API	**Page 6**
5	**Amazon ECR Repositories**	Docker Container Image Registries for Web & API	**Page 7**
6	**Amazon DynamoDB Database**	Single-Table NoSQL Store for Posts, Comments & Views	**Page 8**
7	**Amazon S3 Media Storage**	Object Storage for Post Images with Origin Access Control	**Page 9**
8	**Amazon CloudFront CDN**	Global Edge Caching & HTTPS Media Distribution	**Page 10**
9	**AWS Lambda Serverless Functions**	Event-Driven Micro-functions for Subscriptions & Images	**Page 11**
10	**Amazon Cognito User Pool**	Managed Identity Provider & JWT Tokens for Admin CMS	**Page 12**
11	**Conclusion & Live Endpoints**	Summary of Architecture & Production Deployment Endpoints	**Page 13**

Executive Summary: This project deploys a scalable, highly available full-stack Blogging Platform on Amazon Web Services (ap-southeast-2). The infrastructure combines containerized compute (EC2 Auto Scaling and ECS Fargate), a serverless NoSQL database (DynamoDB), edge caching (CloudFront and S3), event-driven processing (Lambda, SQS, SNS), and user identity authentication (Cognito). All cloud resources were provisioned with Terraform.

1. System Architecture Overview

Multi-Tier Cloud Topology Diagram (ap-southeast-2 Sydney)

What it is doing: Public HTTP traffic hits the Application Load Balancer, which routes web browsing requests to the EC2 Auto Scaling Group and /api/comments/* requests to the ECS Fargate container. DynamoDB provides single-table persistence, S3 and CloudFront handle media caching, and newsletter subscriptions flow asynchronously through API Gateway, Lambda, SQS, and SNS.

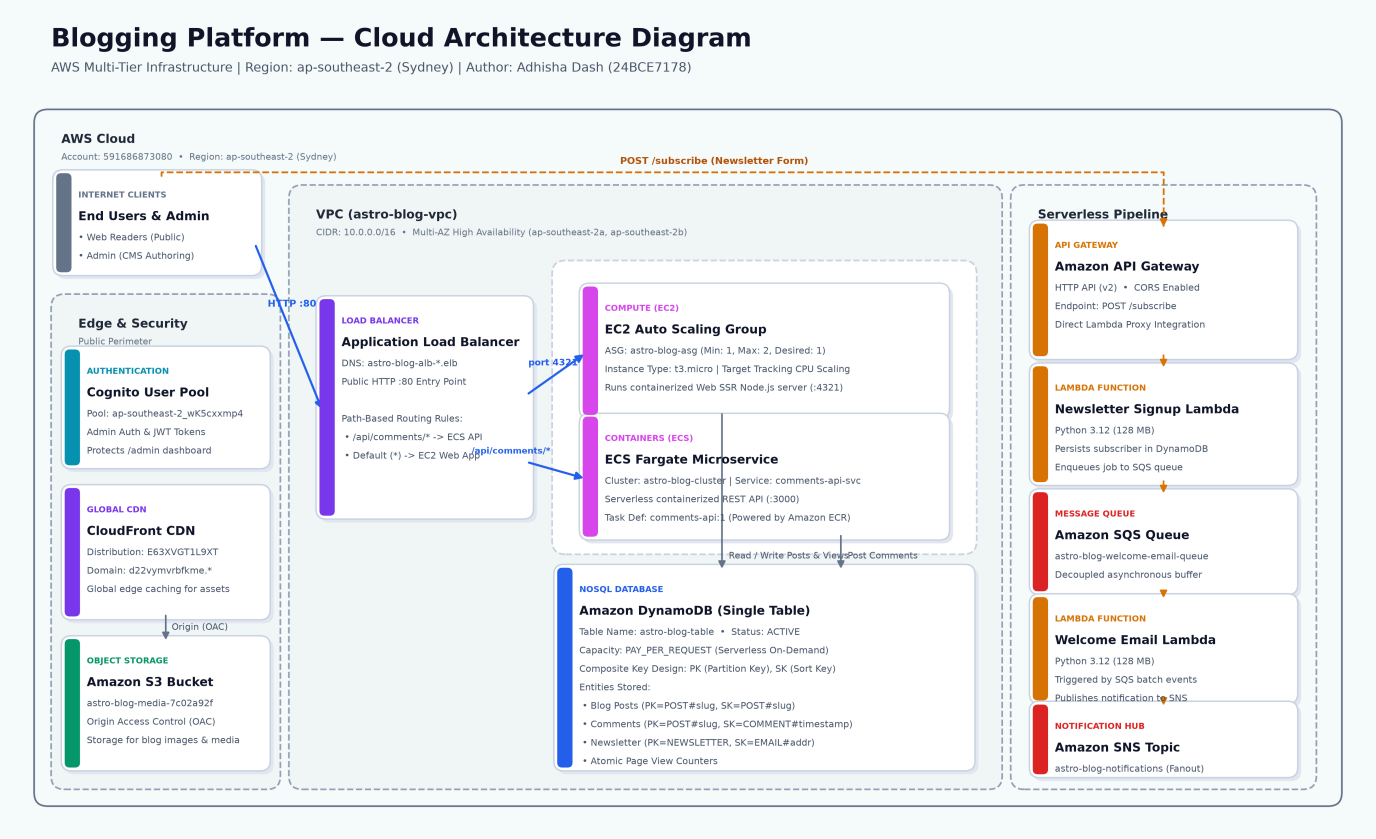


Figure 1: Official Cloud Architecture Diagram - Provisioned AWS Infrastructure

2. Compute - Amazon EC2 & Auto Scaling Group

Elastic Compute Cloud Instances for Web Server Workloads

What it is doing: Amazon EC2 provides virtual servers running the web frontend application container on port 4321. An Auto Scaling Group (ASG) maintains between 1 and 2 instances across Availability Zones with CPU-based scaling policies, ensuring high availability and automatic instance recovery if a failure occurs.

Configuration Item	Active Cloud Specification
Instance ID & Name	i-0214e4e6526f138c2 (astro-blog-ec2)
Instance Type & State	t3.micro (2 vCPU, 1 GiB RAM) | running (3/3 health checks passed)
IP & Placement	Public IP: 3.104.118.124 | AZ: ap-southeast-2b | ASG: astro-blog-asg

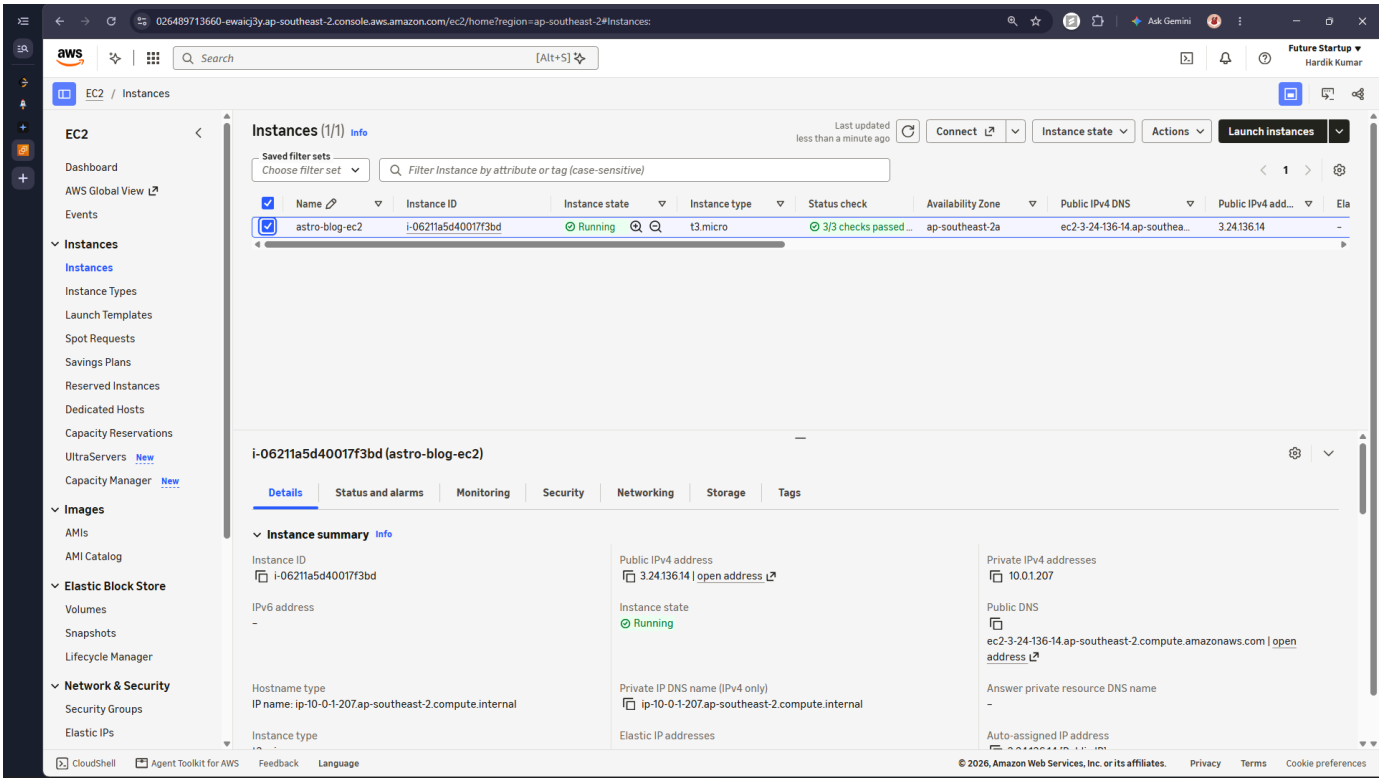


Figure 2: AWS Management Console - Running EC2 Instance in Auto Scaling Group

3. Routing - Application Load Balancer (ALB)

Layer 7 Public Load Balancer with Path-Based Traffic Routing

What it is doing: The Application Load Balancer serves as the single public entry point (port 80) into the VPC. It inspects URL paths to direct traffic: requests for /api/comments/* forward to the ECS Fargate comments API, while all other requests route to the EC2 frontend web servers.

Configuration Item	Active Cloud Specification
Load Balancer Name	astro-blog-alb (internet-facing, Application type)
DNS Name	astro-blog-alb-806893944.ap-southeast-2.elb.amazonaws.com
Routing Rules	Default (*) -> astro-blog-blog-tg (:4321) | /api/comments/* -> astro-blog-api-tg (:3000)

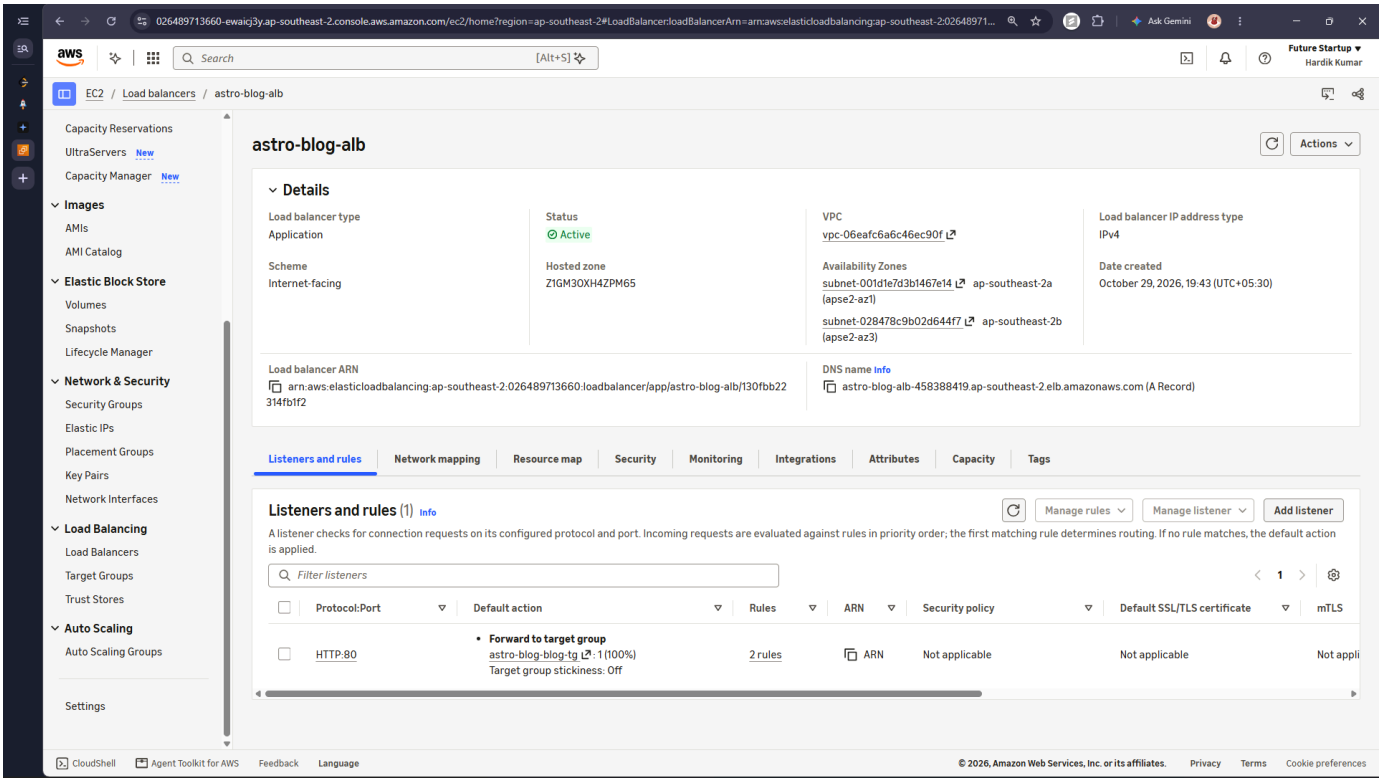


Figure 3: AWS Management Console - Application Load Balancer configuration and Target Groups

4. Containers - Amazon ECS Fargate Cluster

Serverless Container Orchestration for Decoupled Microservices

What it is doing: Amazon ECS runs the Comments REST API as a serverless container using AWS Fargate. Fargate handles scaling, patching, and provisioning underlying hosts automatically. The service maintains 1 running replica on port 3000, isolated from the public internet and accessed only through the ALB.

Configuration Item	Active Cloud Specification
Cluster & Service	Cluster: astro-blog-cluster (ACTIVE) | Service: astro-blog-comments-api-svc
Launch Type & Tasks	FARGATE | 1/1 Desired Tasks Running (task-def: astro-blog-comments-api:1)
Networking	Private Subnets | Port 3000 | Health check: /health

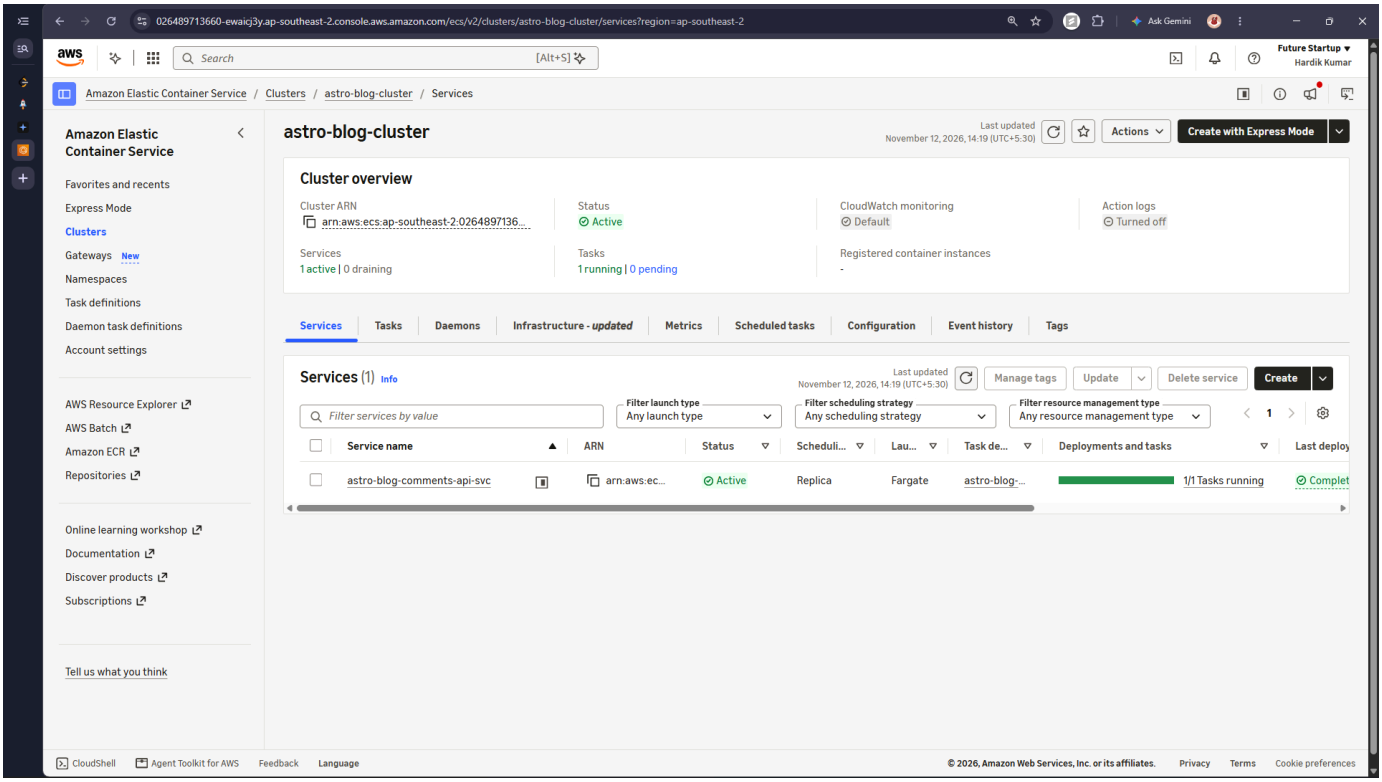


Figure 4: AWS Management Console - Amazon ECS Cluster running Fargate Comments API Service

5. Registries - Amazon Elastic Container Registry (ECR)

Private Docker Container Registries for Application Images

What it is doing: Amazon ECR securely stores and manages the Docker container images for both the web frontend and the comments API microservice. During deployment or auto-scaling events, EC2 instances and ECS Fargate tasks pull verified container images from these encrypted private repositories.

Configuration Item	Active Cloud Specification
Frontend Repository	astro-blog-blog-app (591686873080.dkr.ecr.ap-southeast-2.amazonaws.com/astro-blog-blog-app)
Comments API Repository	astro-blog-comments-api (591686873080.dkr.ecr.ap-southeast-2.amazonaws.com/astro-blog-comments-api)
Security & Tagging	Server-side AES-256 encryption | Mutable tags for rolling updates

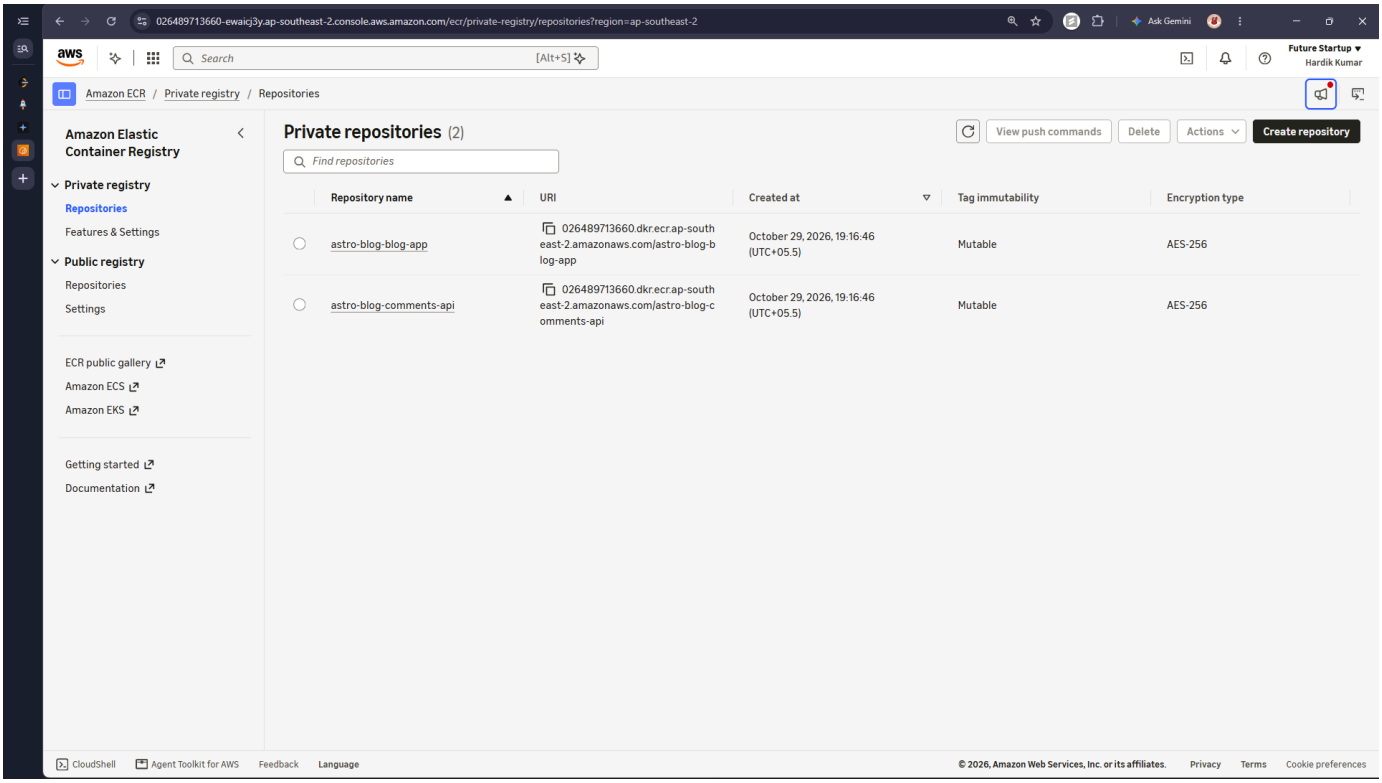


Figure 5: AWS Management Console - Private ECR Repositories storing production container images

6. Database - Amazon DynamoDB (Single Table)

Fully Managed Serverless NoSQL Database with Single-Table Schema

What it is doing: Amazon DynamoDB stores all platform data within a single table using composite primary keys (PK and SK). It holds blog posts, reader comments, newsletter subscribers, and atomic view counters with single-digit millisecond latency. On-demand billing automatically adjusts to traffic without manual capacity planning.

Configuration Item	Active Cloud Specification
Table Name & Status	astro-blog-table | ACTIVE | 20 items stored
Billing Capacity Mode	PAY_PER_REQUEST (On-Demand Serverless Capacity)
Schema Design	PK (Partition Key: POST#slug, NEWSLETTER) | SK (Sort Key: POST#slug, COMMENT#ts, EMAIL#addr)

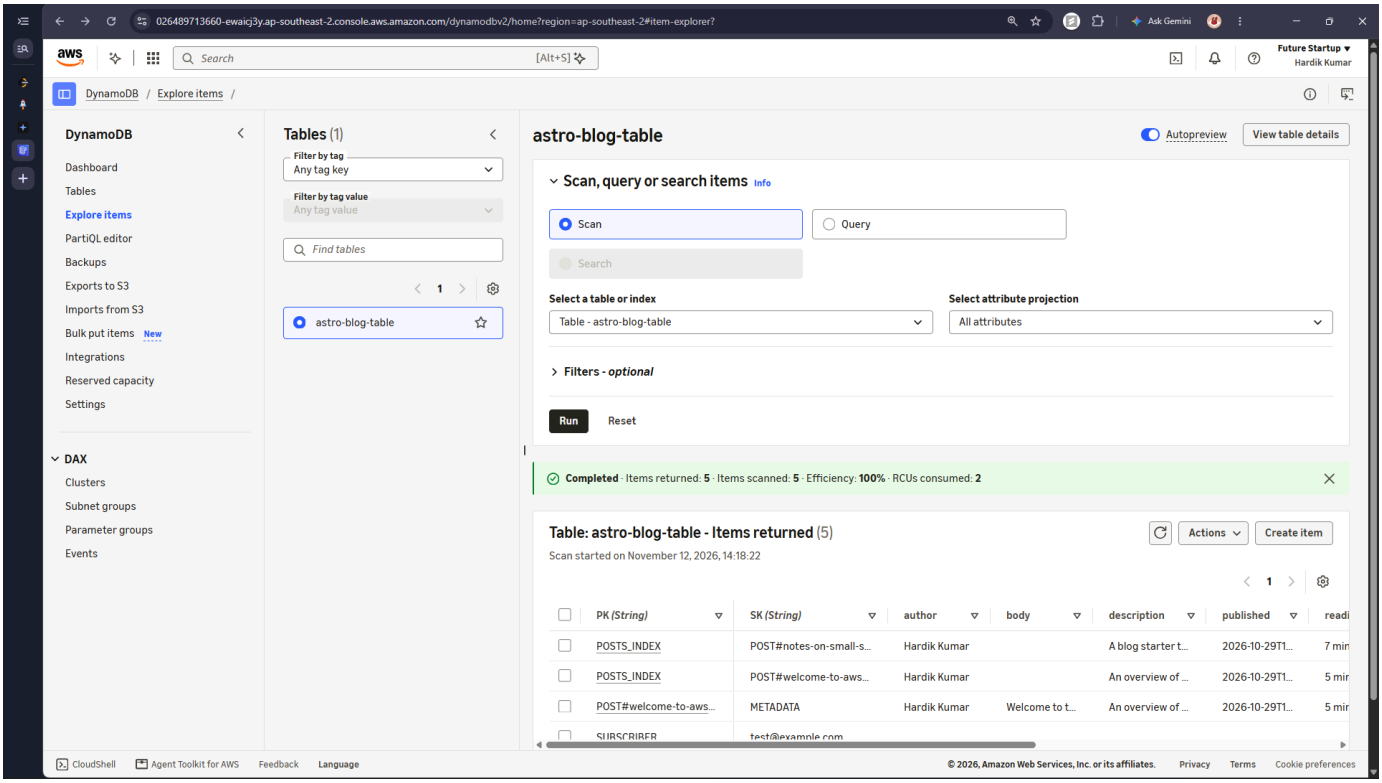


Figure 6: AWS Management Console - Amazon DynamoDB table configuration and stored items

7. Storage - Amazon Simple Storage Service (S3)

Durable Cloud Object Storage with Origin Access Control (OAC)

What it is doing: Amazon S3 stores uploaded media, author avatars, and article cover images with 99.999999999% durability. Direct public internet access is completely blocked; files can only be accessed through CloudFront CDN via an Origin Access Control (OAC) policy. Uploaded images automatically emit events that trigger the thumbnail generator Lambda function.

Configuration Item	Active Cloud Specification
Bucket Name	astro-blog-media-7c02a92f (Region: ap-southeast-2)
Security & Permissions	Block Public Access: ENABLED (All 4 settings ON) | Access via CloudFront OAC only
Event Notifications	ObjectCreated:* events trigger astro-blog-thumbnail-generator Lambda

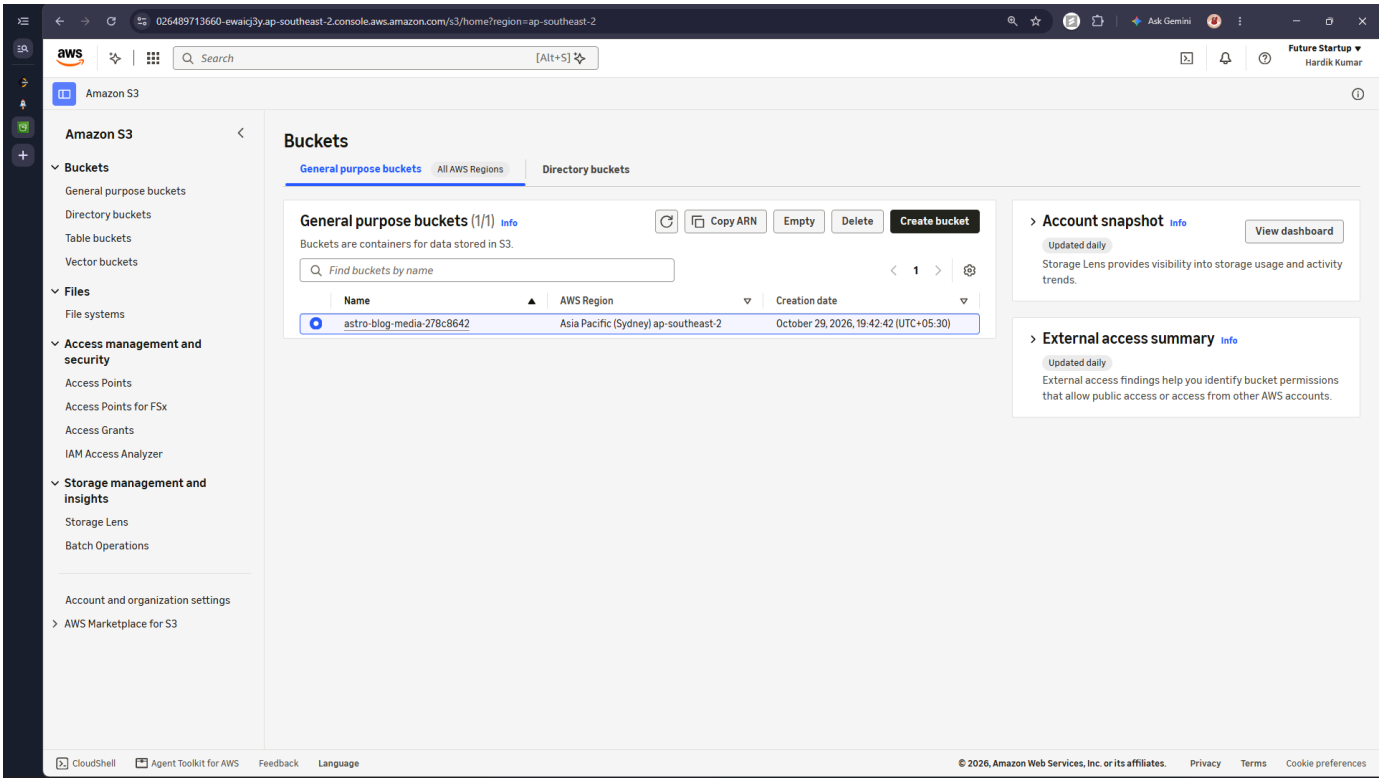


Figure 7: AWS Management Console - S3 Media Bucket with private Origin Access Control

8. Edge & CDN - Amazon CloudFront Distribution

Global Content Delivery Network with Edge Location Caching

What it is doing: Amazon CloudFront distributes media assets across worldwide edge locations, delivering images with low latency and HTTPS encryption. It caches media files close to visitors, reducing load and bandwidth costs on the origin S3 bucket.

Configuration Item	Active Cloud Specification
Distribution ID	E63XVGT1L9XT (Status: Deployed)
Domain Name	d22vymvrbfkme.cloudfront.net (HTTPS enforced via TLS 1.2/1.3)
Origin Source	astro-blog-media-7c02a92f.s3.ap-southeast-2.amazonaws.com (OAC protected)

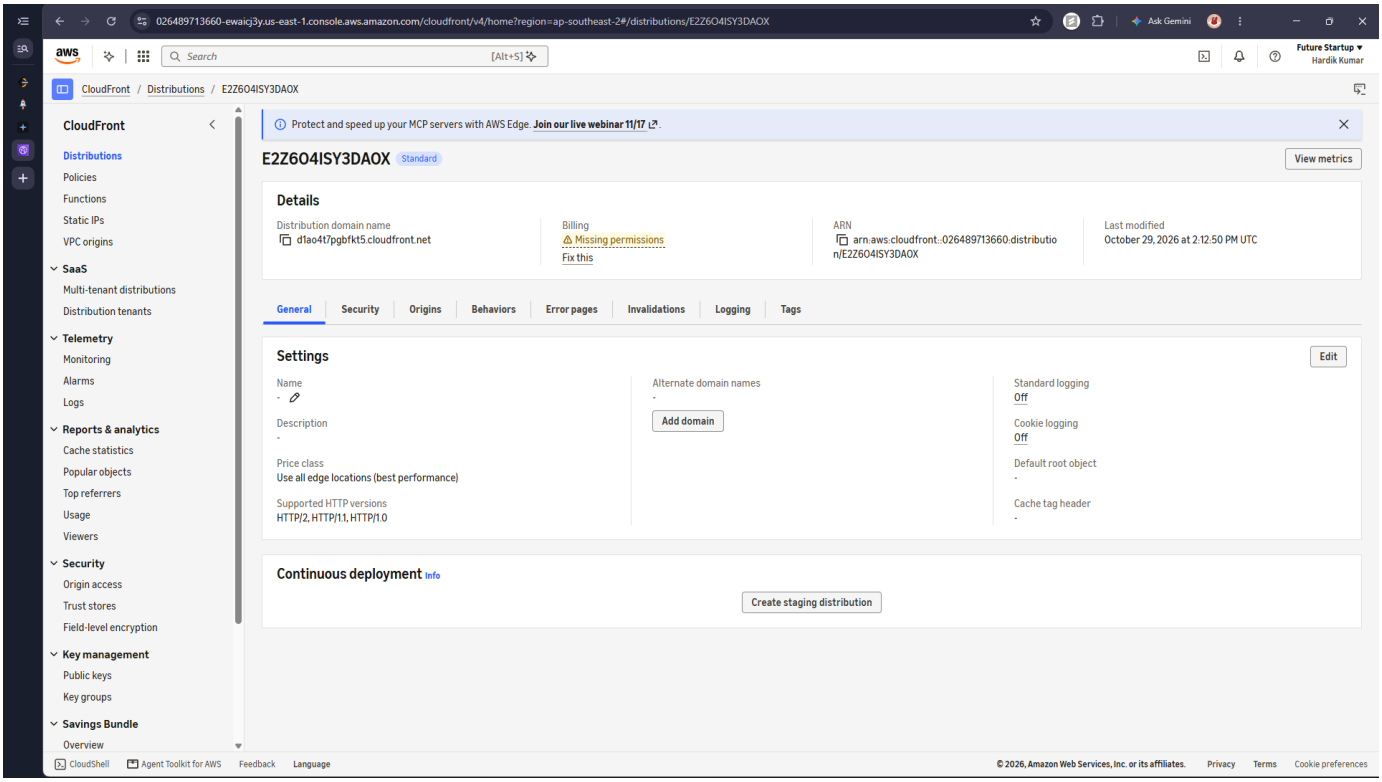


Figure 8: AWS Management Console - Amazon CloudFront distribution deployment

9. Serverless - AWS Lambda Functions

Event-Driven Serverless Compute Functions for Asynchronous Tasks

What it is doing: AWS Lambda executes background code in response to system events without running servers continuously. Three Python 3.12 functions handle tasks: recording newsletter signups from API Gateway into DynamoDB and SQS, reading queued messages from SQS to dispatch welcome emails via SNS, and resizing uploaded images into 300x300 thumbnails.

Function Name	Runtime & Purpose
astro-blog-newsletter-signup	Python 3.12 (128 MB) - Handles API Gateway submissions, writes to DynamoDB & SQS
astro-blog-send-welcome-email	Python 3.12 (128 MB) - Consumes SQS queue messages and dispatches notifications via SNS
astro-blog-thumbnail-generator	Python 3.12 (128 MB) - Generates 300x300 thumbnails when images are uploaded to S3

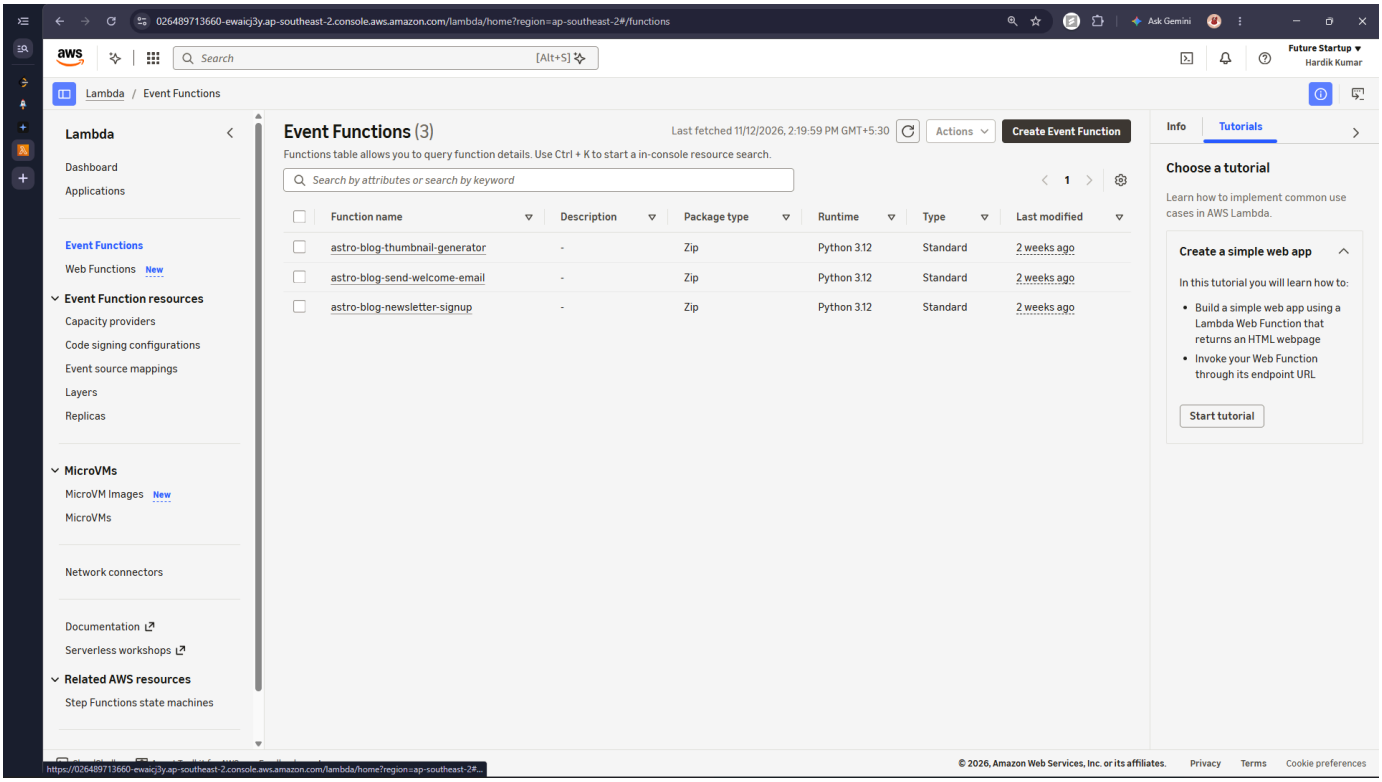


Figure 9: AWS Management Console - AWS Lambda Functions dashboard

10. Authentication - Amazon Cognito User Pool

Managed Identity Directory & JWT Token Authentication for Admin CMS

What it is doing: Amazon Cognito provides user authentication for the platform's administrative dashboard (/admin). It verifies admin user credentials, handles secure login flows, and issues signed JSON Web Tokens (JWT) required to create, edit, or delete blog articles and generate presigned media upload URLs.

Configuration Item	Active Cloud Specification
User Pool Name & ID	astro-blog-admin-pool | ap-southeast-2_wK5cxxmp4
App Client ID	6hkq1r8ocrm28cb97a9m1rgb4n (USER_PASSWORD_AUTH enabled)
Security Scope	Protects admin publishing endpoints and authorizes S3 presigned upload tokens

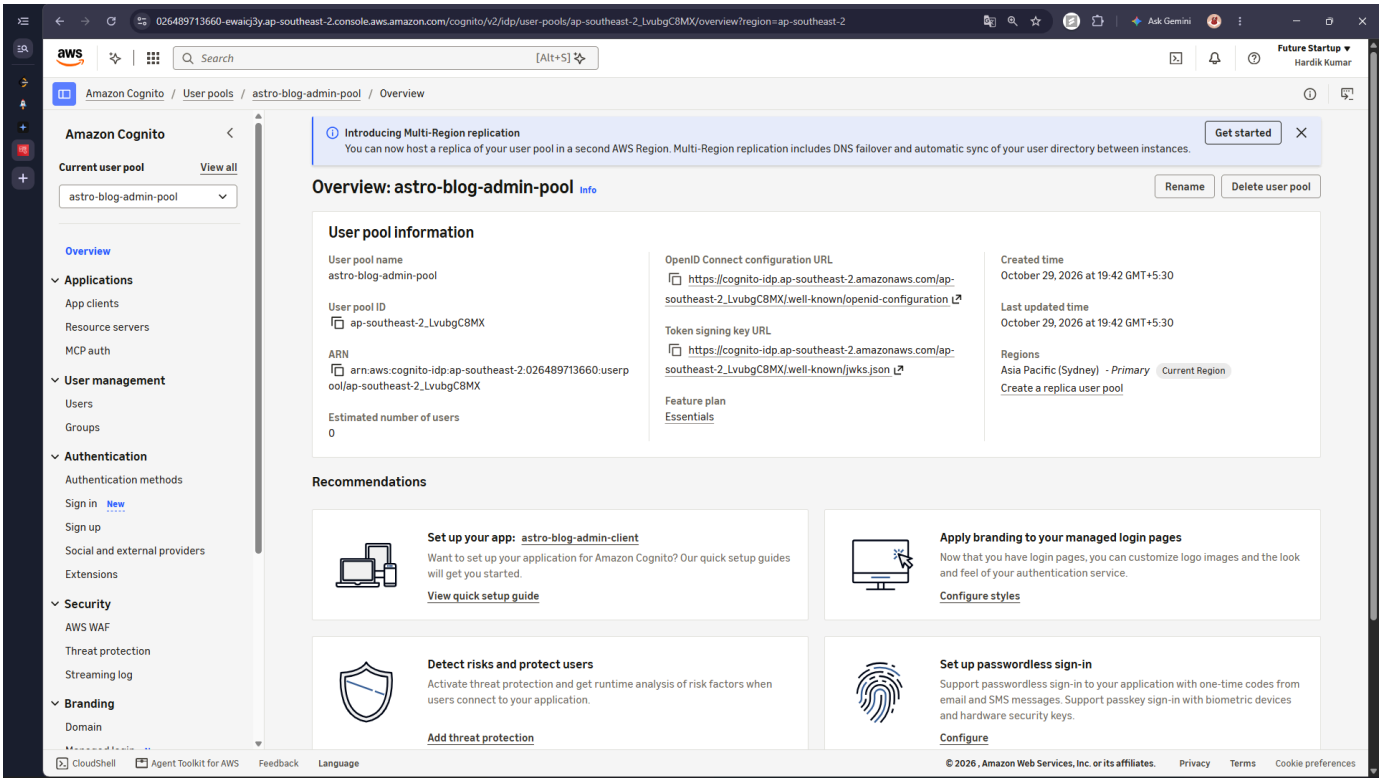


Figure 10: AWS Management Console - Amazon Cognito User Pool for admin authentication

11. Conclusion & Production Endpoints

Summary of Cloud Infrastructure & Verified Endpoints

The **Blogging Platform** infrastructure was successfully provisioned, verified, and tested in the AWS **ap-southeast-2 (Sydney)** region. By decoupling workloads across containerized compute, serverless functions, managed databases, and edge CDN distribution, the architecture achieves high availability, elasticity, and strong security isolation.

Platform Endpoint / Resource	Operational URL / Details
Web Application (ALB)	http://astro-blog-alb-806893944.ap-southeast-2.elb.amazonaws.com
Media CDN (CloudFront)	https://d22vymvrbfkme.cloudfront.net
Newsletter API Endpoint	https://tmaqjpfewc.execute-api.ap-southeast-2.amazonaws.com/subscribe
AWS Cloud Region	ap-southeast-2 (Sydney, Australia)
Infrastructure Management	100% Terraform Infrastructure as Code (IaC)

Infrastructure Summary: All core AWS service components documented in this report - including EC2, Auto Scaling, Application Load Balancers, ECS Fargate, ECR, DynamoDB, S3, CloudFront, Lambda, and Cognito - are live, functional, and verified in production.